Contents

Go Best Practices Guide

This document explains the intermediate Go concepts and coding conventions used throughout this codebase. It is designed for developers who know basic Go but want to understand the idioms and patterns used here.

> **Note:** This codebase follows Go best practices and idiomatic patterns. For explanations of intermediate Go concepts used in the code, see GO_CONCEPTS.md. This guide provides deeper explanations and context.

Table of Contents

Error Handling

The Go Error Pattern

In Go, errors are values, not exceptions. Functions return errors as the last return value:

```go
// Standard Go pattern: (result, error) or (result1, result2, error)
func GetMetrics() (*Metrics, error) {
    // ... code ...
    if err != nil {
        return nil, err  // Return nil result and the error
    }
    return metrics, nil  // Return result and nil error
}
```

Why this pattern? - Forces explicit error handling (no hidden exceptions) - Makes error handling visible in the code - Allows multiple return values (result + error)

Checking Errors

Always check errors immediately:

```
//  GOOD: Check error immediately
metrics, err := GetMetrics()
if err != nil {
    return nil, err  // Handle or propagate
}
// Use metrics here

//  BAD: Ignoring errors
metrics, _ := GetMetrics()  // Do not do this!
```

Best Practice: Never ignore errors. If you must, document why with a comment.

Pointers and Value Semantics

When to Use Pointers

Use pointers (`*Type`) when: 1. The value is large (structs with many fields) 2. You need to modify the value 3. The value can be `nil` (optional) 4. You want to avoid copying (performance)

Use values when: 1. The value is small (primitives, small structs) 2. You do not need to modify it 3. Immutability is desired

Examples in This Codebase

```
// Pointer receiver - modifies the struct, avoids copying
func (c *EnlightenCloudClient) GetMetrics() (*LocalMetrics, error) {
    // 'c' is a pointer - changes to c affect the original
}

// Value return - small struct, no modification needed
func getDefaultColors() ColorConfig {
    return ColorConfig{...}  // Return by value
}

// Pointer return - large struct, may be nil
func LoadConfig() (*Config, error) {
    config := &Config{}  // Create pointer
    return config, nil   // Return pointer
}
```

Struct Methods and Receivers

Receiver Types

Go methods are functions with a special “receiver” parameter:

```go
// Value receiver - receives a copy
func (d Display) ShowInfo(msg string) {
    // Changes to 'd' do not affect the original
}

// Pointer receiver - receives a reference
func (c *EnlightenCloudClient) GetMetrics() (*LocalMetrics, error) {
    // Changes to 'c' affect the original
    c.cacheUsed = true  // Modifies the original struct
}
```

When to Use Each

Use pointer receivers (`*Type`) when: - Method modifies the struct - Struct is large (avoids copying) - Consistency (if any method uses pointer, all should)

Use value receivers (`Type`) when: - Method does not modify the struct - Struct is small - You want immutability

In this codebase: We use pointer receivers for all struct methods because: 1. Methods often modify state (e.g., `cacheUsed` flag) 2. Structs are moderately sized 3. Consistency across the codebase

---

Channels and Goroutines

> **For a comprehensive deep dive into channels, signals, and the `select` statement as used in this codebase, see GO_CONCEPTS.md**. This section provides a brief overview of the concepts.

Channels

Channels are Go's way to communicate between goroutines safely. In this codebase, we use `signal.NotifyContext` which creates a context that is cancelled when signals are received.

Channel Types: - `chan T` - bidirectional (can send and receive) - `chan<- T` - send-only - `<-chan T` - receive-only

Select Statement

`select` lets you wait on multiple channel operations:

```go
select {
case <-ticker.C:
    // Timer ticked - do periodic work
case <-ctx.Done():
    // Signal received - handle shutdown
    return
}
```

Why use select? - Non-blocking: waits for first available case - Allows handling multiple channels - Essential for graceful shutdown

For detailed examples, real-world scenarios, and edge cases, see GO_CONCEPTS.md.

JSON Struct Tags

The Problem

Go uses PascalCase for exported fields, but JSON APIs often use snake_case:

```
type SystemConfig struct {
    Name string // Go convention: PascalCase
    ID   string
}
// JSON API returns: {"name": "My System", "id": "12345"}
```

The Solution: Struct Tags

Struct tags tell the JSON encoder/decoder how to map fields:

```
type SystemConfig struct {
    Name string `json:"name"` // Maps to "name" in JSON
    ID   string `json:"id"`   // Maps to "id" in JSON
}
```

Tag Syntax: - Backticks: `` `json:"field_name"` `` - Multiple tags: `` `json:"field_name" xml:"FieldName"` `` - Omit if empty: `` `json:"field_name,omitempty"` ``

Common Tags in This Codebase

```
type TelemetryResponse struct {
    LastReportedAggregateSOC string              `json:"last_reported_aggregate_soc,omitempty"
    Intervals                []TelemetryInterval `json:"intervals"`
}
```

Defer Statements

What is Defer?

`defer` schedules a function call to execute when the surrounding function returns:

```
func ReadFile(filename string) ([]byte, error) {
    file, err := os.Open(filename)
    if err != nil {
        return nil, err
    }
    defer file.Close()  // Always closes, even if function returns early
```

```
    // ... read file ...
    return data, nil  // file.Close() executes here
}
```

Defer Execution Order

Defer calls execute in LIFO (Last In, First Out) order:

```
defer fmt.Println("First")
defer fmt.Println("Second")
defer fmt.Println("Third")
// Output when function returns:
// Third
// Second
// First
```

Common Uses

1. **Resource cleanup:**

   ```
   resp, err := http.Get(url)
   if err != nil {
       return err
   }
   defer resp.Body.Close()  // Always close response body
   ```

2. **Stopping timers:**

   ```
   ticker := time.NewTicker(interval)
   defer ticker.Stop()  // Always stop ticker
   ```

Slice Capacity Hints

The Problem

When you know how many elements a slice will hold, pre-allocating capacity is more efficient:

```
//  SLOW: Slice grows multiple times
systems := []SystemMetrics{}
for _, sys := range config.Systems {
    systems = append(systems, metrics)  // May reallocate multiple times
}

//  FAST: Pre-allocate capacity
systems := make([]SystemMetrics, 0, len(config.Systems))
//                               ^length ^capacity
for _, sys := range config.Systems {
    systems = append(systems, metrics)  // No reallocation needed
}
```

How It Works

```
// make([]Type, length, capacity)
systems := make([]SystemMetrics, 0, 10)
//                                ^  ^
//                                |     Can hold 10 elements without reallocating
//                                  Currently has 0 elements
```

In this codebase:

```
// aggregator.go - we know exactly how many systems we have
Systems: make([]SystemMetrics, 0, len(systems)),
//                              ^  ^
//                              |     Capacity = number of systems
//                                Start with empty slice
```

Interface Satisfaction

Implicit Interfaces

Go interfaces are satisfied **implicitly** - no `implements` keyword needed:

```
// io.Reader interface (from standard library)
type Reader interface {
    Read(p []byte) (n int, err error)
}

// Any type with a Read method automatically satisfies io.Reader
type MyReader struct{}

func (r MyReader) Read(p []byte) (n int, err error) {
    // ... implementation ...
}
// MyReader now satisfies io.Reader - no explicit declaration needed!
```

Why This Matters

```
// http.Response.Body is an io.Reader
// We can pass it to any function expecting io.Reader
func ReadAll(r io.Reader) ([]byte, error) {
    // Works with http.Response.Body, os.File, bytes.Buffer, etc.
}

resp, _ := http.Get(url)
data, _ := ReadAll(resp.Body)  // resp.Body satisfies io.Reader
```

Benefits: - Loose coupling: functions depend on behavior, not types - Easy testing: can create mock implementations - Flexible: types can satisfy multiple interfaces

---

Package-Level Variables

What Are They?

Variables declared outside functions at package level:

```
package main

// Package-level variables
var (
    tokenCache *TokenCache  // Shared cache
)
```

When to Use

Use package-level variables for: - Shared state across functions (caches, configuration) - Configuration that does not change - Constants

Avoid for: - Data that should be passed as parameters - State that should be encapsulated in structs

In This Codebase

```
// oauth.go - token cache shared across all OAuth operations
var (
    tokenCache *TokenCache  // Shared cache (accessed from main goroutine only)
)

// api_cache.go - configuration flags
var (
    testMode      bool  // When true, only use cached responses (no live API calls)
    cacheDisabled bool  // When true, always make live API calls
)
```

Why package-level here? - These are singletons (one cache, one set of flags) - Used across multiple functions - Need to persist between function calls - In this codebase, all shared state is accessed from the main goroutine only (single-threaded execution)

Error Wrapping

The Problem

When propagating errors, you want to add context without losing the original error:

```
//  BAD: Loses original error
if err != nil {
    return fmt.Errorf("failed to get metrics")  // Original error lost!
}

//  GOOD: Wraps original error
if err != nil {
```

```
    return fmt.Errorf("failed to get metrics: %w", err)  // Preserves original
}
```

The %w Verb

The `%w` verb in `fmt.Errorf` wraps errors, preserving the error chain:

```
func GetMetrics() (*Metrics, error) {
    data, err := fetchData()
    if err != nil {
        // Wrap with context, preserve original error
        return nil, fmt.Errorf("failed to fetch data: %w", err)
    }
    return parseData(data)
}

// Caller can inspect the error chain:
metrics, err := GetMetrics()
if err != nil {
    // Can check for specific error types
    if errors.Is(err, io.EOF) {
        // Handle EOF specifically
    }
    // Or unwrap to get original error
    originalErr := errors.Unwrap(err)
}
```

Error Chain Example

```
// api_cache.go
if err != nil {
    return nil, fmt.Errorf("failed to read cache file: %w", err)
}

// cloud_client.go
if err != nil {
    return nil, fmt.Errorf("failed to get metrics: %w", err)
}

// aggregator.go
if err != nil {
    return nil, fmt.Errorf("failed to get metrics from Cloud API: %w", err)
}

// Error chain when it reaches main.go:
// "failed to get metrics from Cloud API: failed to get metrics: failed to read cache file: fi
```

Time Handling

Time Types

Go has several time-related types:

```
time.Time      // Represents a point in time
time.Duration  // Represents a duration (nanoseconds)
*time.Time     // Pointer to time (allows nil = "not set")
```

Common Patterns

1. Creating time values:

```
now := time.Now()                    // Current time
midnight := time.Date(2026, 1, 15, 0, 0, 0, 0, time.UTC)
duration := 5 * time.Minute          // Duration literal
```

2. Parsing dates:

```
// Go's reference time: Mon Jan 2 15:04:05 MST 2006
//                     = 01/02 03:04:05PM '06 -0700
date, err := time.Parse("2006-01-02", "2026-01-15")
```

3. Formatting dates:

```
t := time.Now()
fmt.Println(t.Format("2006-01-02"))           // "2026-01-15"
fmt.Println(t.Format("Mon Jan 2, 2006"))      // "Wed Jan 15, 2026"
fmt.Println(t.Format("03:04:05 PM"))          // "02:30:45 PM"
```

4. Timezone handling:

```
utc := time.Now().UTC()
reportTZ, _ := LoadTimezone(config.Timezone) // From config, system, or US/Pacific fallback
local := time.Now().In(reportTZ)
```

Pointer to Time (Optional Time)

```
// *time.Time allows nil = "not set"
type AggregatedMetrics struct {
    Timestamp time.Time   // Always has a value
    QueryDate *time.Time  // Can be nil (means "today")
}

// Usage:
var queryDate *time.Time  // nil = use today
if userSpecifiedDate {
    d := time.Date(2026, 1, 15, 0, 0, 0, 0, time.UTC)
    queryDate = &d  // Set to specific date
}
```

Coding Conventions Used in This Codebase

This section summarizes the coding conventions and best practices followed throughout this codebase.

1. Error Handling

- **Always check errors immediately** - Do not ignore them
- **Wrap errors with context** - Use `%w` verb in `fmt.Errorf()` to preserve error chain
- **Return errors, do not log and continue** - Let callers decide how to handle errors
- **Fail fast** - Return errors early rather than continuing with invalid state

2. Naming Conventions

- **Exported** (public): PascalCase (`GetMetrics`, `Config`, `SystemMetrics`)
- **Unexported** (private): camelCase (`getCacheKey`, `tokenCache`, `parseResponse`)
- **Constants**: PascalCase or ALL_CAPS (`Reset`, `Bold`, `CacheDir`)
- **Interfaces**: Usually end with `-er` (`Reader`, `Writer`, `Closer`)
- **Constructors**: Prefix with `New` (`NewDisplayWithColorsAndTimezone`, `NewEnlightenCloudClient`)

3. Function Organization

- **Small, focused functions** - One responsibility per function
- **Functions return early on errors** - Reduces nesting, improves readability
- **Exported functions have doc comments** - Explain purpose, parameters, return values
- **Complex logic has inline comments** - Explain "why", not just "what"

4. Comments

- **Package comments** - Explain the package's purpose (first comment in file)
- **Exported functions** - Doc comments starting with function name
- **Complex logic** - Inline comments explaining the reasoning
- **Go concepts** - See GO_CONCEPTS.md for explanations of intermediate Go concepts used in the code

5. Struct Design

- **Group related fields** - Logical organization
- **Use meaningful field names** - Self-documenting code
- **Add JSON tags for API structs** - Map to API field names
- **Document exported structs** - Explain purpose and usage

6. Resource Management

- **Always use `defer` for cleanup** - Files, HTTP bodies, timers
- **Close resources explicitly** - Do not rely on garbage collection
- **Stop timers and tickers** - Prevent resource leaks

7. Pointers vs Values

- **Use pointers for:**
  - Large structs (avoid copying)
  - When you need to modify the value
  - Optional values (nil = not set)
  - Consistency (if any method uses pointer receiver, all should)
- **Use values for:**
  - Small primitives (int, string, bool)
  - When you do not need to modify
  - When immutability is desired

8. Error Wrapping

- **Always use `%w` verb** - Preserves error chain for debugging
- **Add context at each level** - "failed to X: %w" pattern
- **Do not wrap unnecessarily** - Only add context that is useful

9. Testing

- **Test files end with `_test.go`** - Go test runner convention
- **Test functions start with `Test`** - `func TestFunctionName(t *testing.T)`
- **Use table-driven tests** - Test multiple cases in one function
- **Validate against expected values** - This codebase uses `internal/validation/validation.go`

Test File Organization This codebase uses two test file organization patterns:

Standard Pattern (1:1 mapping) Most packages follow the simple convention: - `config.go` → `config_test.go` - `constants.go` → `constants_test.go` - `cli.go` → `cli_test.go`

Complex Pattern (1:many mapping) For packages with many functions or complex logic, tests are split by **test category**:

Cache Package (`cache.go`): 1. **`cache_test.go`** - State management tests - Tests state flags (testMode, cacheDisabled, rateLimitWarning) - Tests ResetState() for test isolation

2. **`cache_functions_test.go`** - Core functionality tests
   - Tests URL redaction, cache saving/loading, normalization
   - Tests file operations, error handling
   - Original functional tests

Why split? Cache has two distinct concerns: state management vs core caching functions.

OAuth Package (`oauth.go`): 1. **`oauth_test.go`** - Basic unit tests (270 lines) - Tests GetAuthorizationURL validation - Tests token refresh mechanics - Original tests from early rounds

2. **`oauth_functional_test.go`** - Integration/functional tests (598 lines)
   - Uses mock HTTP servers (`httptest.NewServer`)
   - Tests complete token exchange flows
   - Tests real HTTP interactions with mocked backend
   - Created in Rounds 4-6 for comprehensive coverage

3. `oauth_edge_cases_test.go` - Edge case & error path tests (442 lines)
    - Tests validation errors (missing config, empty fields)
    - Tests network errors, timeouts, malformed responses
    - Created in Round 10 Phase A to improve coverage

Why split? OAuth is complex (270 lines of implementation): - Basic validation separate from integration tests - Happy path (functional) separate from error paths (edge cases) - Easier to maintain and understand test intent

Benefits of This Approach:

Clarity - Test file name indicates test category **Maintainability** - Related tests grouped together **Readability** - Smaller files easier to navigate (161-598 lines vs 516-1310 lines) **History** - Shows evolution (original → functional → edge cases) **Focus** - Can run specific test categories independently

Bottom line: The 1:many pattern emerged naturally during refactoring to keep test files organized and maintainable as coverage improved from 29.8% → 70.4%. It's a pragmatic choice, not a strict rule.

Additional Resources

- Effective Go - Official Go best practices
- Go Code Review Comments - Common style guide
- Go by Example - Practical examples